



Segundo parcial – 6 de junio de 2025

Nombre y Apellido: VALENTINA
ACEVEDO FRÍAS

Calificación: Aprobado (P.P.)

IMPORTANTE: Resolver cada ejercicio en una hoja separada, indicando claramente el número del ejercicio. Colocar en cada hoja su nombre y apellido. Puede resolver en lápiz solo si no hay borradores y el texto se lee fácilmente sobre la hoja.

Ej 1- Dado un árbol binario donde cada nodo almacena un carácter y un número entero positivo, hallar en único recorrido la cantidad de nodos de grado 1 ó 0 donde el carácter no sea dígito, y la cantidad de nodos donde el número entero sea de 2 dígitos.

Ej 2- (Utilizar TDA N-ario) Dado un árbol N-ario de enteros determinar, mediante una función int, si para todo nodo positivo no hoja de grado par ubicado en un nivel menor a K (dato), se verifica que el promedio de las claves de sus hijos es al menos el doble de la clave del nodo analizado. Si la solución es void, el puntaje obtenido será no mayor a 1,5p

Ej 3- Dado un digrafo con aristas ponderadas almacenado en una lista de adyacencia, en un único recorrido obtener la cantidad de vértices con bucle que tienen al menos una arista de salida con costo menor al costo del bucle y, si existe, eliminarla. No eliminar más de una arista por vértice.

Ej 4-

- a. Dada la siguiente matriz de adyacencia de un digrafo de 5 vértices [A..E] hallar el camino mínimo que toma a C como origen hacia el resto de los vértices, obteniendo además el camino a cada uno de ellos. Mostrar paso a paso la aplicación del algoritmo seleccionado. Indicar qué algoritmo aplicó.

M =

	A	B	C	D	E	
A	0	9	5	0	2	A
B	3	2	0	0	3	B
C	0	9	0	3	0	C
D	1	5	8	0	9	D
E	4	0	7	5	0	E

- b. En un árbol AVL inicialmente vacío, insertar paso a paso las siguientes claves: 21, 25, 30, 11, 70, 8, 14, 23

Comprobar adecuadamente **tras cada inserción** si el árbol resultante es o no AVL.

A partir del árbol final, indicar si existe alguna clave no hoja que al eliminarla el árbol deje de ser AVL, **justificando** la respuesta.

Serán considerados al calificar este examen la eficiencia de las soluciones y el uso de las características del lenguaje C y de los ppios de la programación estructurada. En los ej 1 a 3, desarrollar un main.c completo que incluya declaraciones, inicializaciones, invocaciones y resultados obtenidos. En los ejercicios 1 y 3, declarar tipos.

Para **aprobar** es necesario obtener al menos 5p; 4p de los cuales deben obtenerse entre los ej 1 a 3

Para acceder al coloquio de **promoción** es necesario obtener al menos 6p y obtener al menos el 50% de c/ej

Ej 1 (2p)	Ej 2 (3p)	Ej 3 (3p)	Ej 4 (2p)	FINAL
2	2,25	2,25	1,75	8,25

1)

INT ES Dígito (char car) {

return car >= '0' && car <= '9';

}

INT Tiene2Dig (unsigned int num) {

return num >= 10 && num <= 99;

}

void HallarCants (Arbol A, INT * NodosA, INT * NodosB) {

if (A != NULL) {

if ((A->izq == NULL && A->der == NULL || A->izq != NULL
&& A->der != NULL || A->izq != NULL && A->der == NULL
&& ! es Dígito (A->dato.car))

der. izq nulo || der nulo

(* NodosA) ++;

if (Tiene2Dig (A->dato.num))

(* NodosB) ++;

HallarCants (A->izq, NodosA, NodosB);

HallarCants (A->der, NodosA, NodosB);

}

}


```
#include <stdio.h>
#include <stdlib.h>
```

```
typedef struct t
```

```
    unsigned int num;
    char car;
```

```
    t *der;
```

} Tipo innecesario

```
typedef struct nodo t
```

```
    t *der;
```

```
    struct nodo *izq, *der;
```

```
    nodo;
```

```
typedef struct nodo *Arbol;
```

```
int main () {
```

```
    Arbol A;
```

```
    int nodosA = 0, nodosB = 0;
```

```
    cargaArbol (&A);
```

```
    HallarCants (A, &nodosA, &nodosB);
```

```
    printf ("Hay %d nodos de grado 1 o 0 donde el caracter  
no es digito", nodosA);
```

```
    printf ("Hay %d nodos donde el numero entero es de 2  
digitos", nodosB);
```

```
    return 0;
```

```
}
```



```

②
INT verifica ( Arbol A, Pos P, INT k, INT nivel ) {
    Pos C;
    INT cumple = 1, grado = 0, acumClav = 0;
    IF ( !NULO(P) && !NULO(HijoMasIzq(P, A)) && nivel < k ) {
        C = HijoMasIzq(P, A);
        IF ( INFO(P, A) <= 0 ) {
            while ( !NULO(C) && cumple ) {
                cumple = verifica(A, C, k, nivel + 1);
                C = HermanoDer(C, A);
            }
            return cumple;
        }
        else {
            while ( !NULO(CC) && cumple ) {
                cumple = verifica(A, C, k, nivel + 1);
                grado ++;
                acumClav += INFO(C, A);
                C = HermanoDer(C, A);
            }
            return cumple && grado * 2 == 0 &&
                acumClav >= 2 * INFO(P, A) * grado;
        }
    }
    else {
        return 1;
    }
}

```

Si el grado es impar
devuelve 0!!

No es correcto.
Solo evaluar el
promedio si se
cumplen las
condiciones.

* No calcula el promedio


```
#include <stdio.h>
#include "arbol.h"
```

```
int main () {
```

```
    Arbol A;
    int k;
```

```
    cargaArbol (&A);
```

```
    printf("Ingrese un nivel /n");
    scanf("%d", &k);
```

```
    if (verifica(A, raiz(A), k, 1))
```

```
        printf("el arbol cumple que");
```

```
    else
```

```
        printf("el arbol no cumple que");
```

```
    printf("Todo nodo positivo no hoja de grado par ubicado a  
un nivel menor a %d verifica que el promedio de las  
claves de sus hijos es al menos el doble de la clave del  
nodo analizado", k);
```

```
    return 0;
```

```
}
```


(3)

void E3(TLISTA A[], INT n, INT *cantVert) {

INT min, i, eliminar = 0, hayBucle = 0;
TLISTA ant, antMin, act;

for (i = 0, i < n, i++) {

act = A[i], ant = NULL;

min = INT_MAX; ??

while (act != NULL && !eliminar) {

if (act->vert == i) {

hayBucle = 1;

if (act->costo > min)

eliminar = 1;

else

min = act->costo;

if

else

if (act->costo < min) {

min = act->costo;

antMin = ant;

eliminar = hayBucle;

if

ant = act;

act = act->sig;

if

*cantVert += hayBucle && eliminar;

if (eliminar) {

if (antMin == NULL) {

act = A[i]->sig;

A[i]->sig = act->sig;

if

else

act = antMin->sig;

antMin->sig = act->sig;

if

free(act);

if

La inicialización de eliminar
debe hacerse por vertice, sino
se rompe en 5

MAZ

A[i] = A[i]->sig;
Pierde un nodo.

11

40

#include <stdio.h>
 #include <stdlib.h> *also include limits.h*
 #define DIM 50

typedef struct nodo {

int costo, vert;
 struct nodo *sig;

} nodo;

typedef nodo * TLista;

int main () {

TLista A[DIM];

int n, cantvert = 0;

cargaLista(A, n);

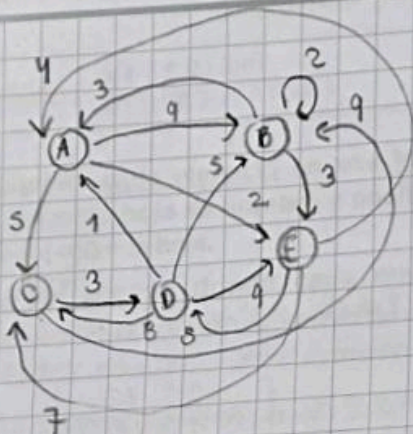
EJ3(A, n, &cantvert);

printf("%d vertices con bucle tienen al menos una arista
 de salida con costo menor al costo del bucle", cantvert);

return 0;

Aplico el algoritmo de Dijkstra.

a)



② $S = \{C\}$

$$D = \begin{bmatrix} \infty & 9 & 0 & 3 & \infty \end{bmatrix}$$

$$P = \begin{bmatrix} \infty & C & C & C & \infty \end{bmatrix}$$

③ $S = \{C, D\}$

$$D[A] = \min(\infty, 3 + 1) = 4$$

$$D[B] = \min(9, 3 + 5) = 8$$

$$D[E] = \min(\infty, 3 + 9) = 12$$

$$D = \begin{bmatrix} 4 & 8 & 0 & 3 & 12 \end{bmatrix}$$

$$P = \begin{bmatrix} D & D & C & C & D \end{bmatrix}$$

④ $S = \{C, D, A\}$

$$D[B] = \min(8, 4 + 9) = 8$$

$$D[E] = \min(12, 4 + 2) = 6$$

$$D = \begin{bmatrix} 4 & 8 & 0 & 3 & 6 \end{bmatrix}$$

$$P = \begin{bmatrix} D & D & C & C & A \end{bmatrix}$$

⑤ $S = \{C, D, A, E\}$

$$D[B] = \min(8, 6 + \infty) = 8$$

$$D = \begin{bmatrix} 4 & 8 & 0 & 3 & 6 \end{bmatrix}$$

$$P = \begin{bmatrix} D & D & C & C & A \end{bmatrix}$$

⑥ $S = \{C, D, A, E, B\}$

Se ingresaron todas las aristas a S, fin del algoritmo.

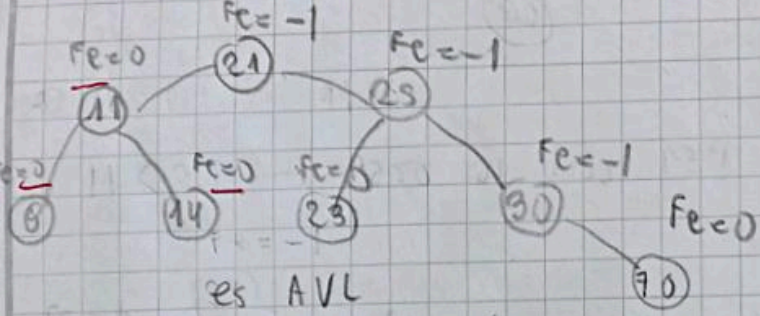
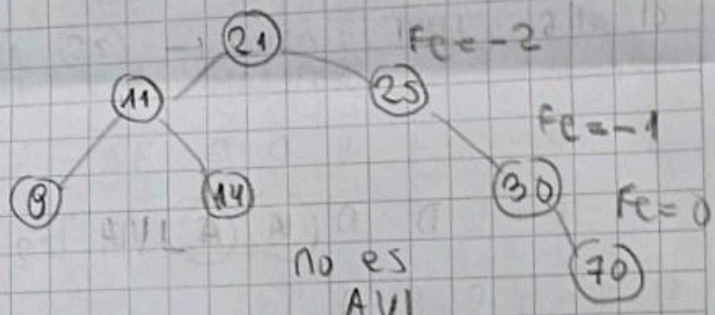
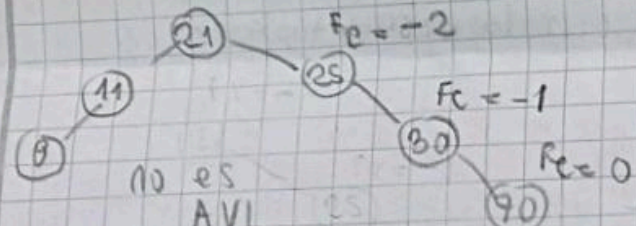
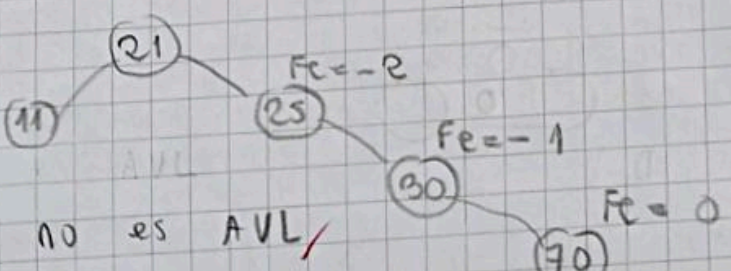
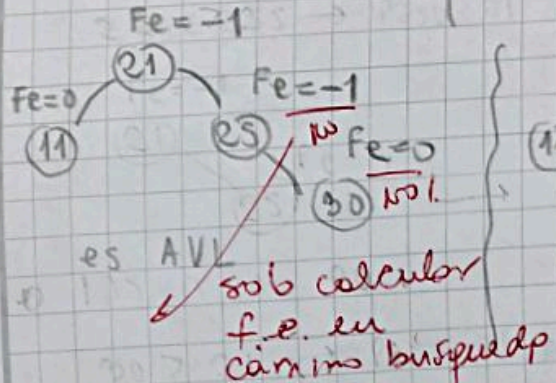
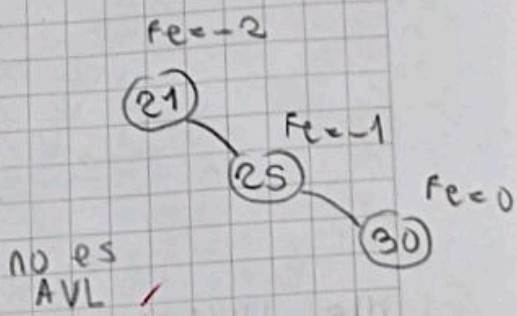
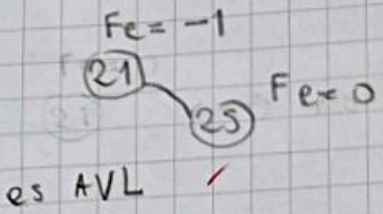
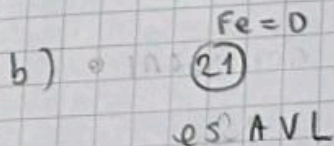
Caminos mínimos.

C $\xrightarrow{3}$ D $\xrightarrow{1}$ A ✓

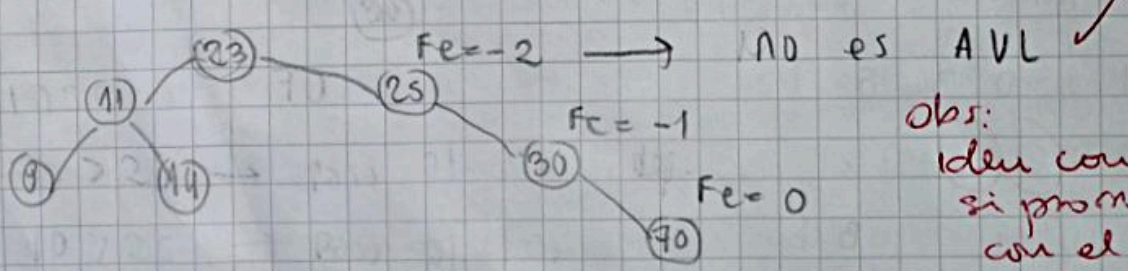
C $\xrightarrow{3}$ D $\xrightarrow{3}$ B ✓

C $\xrightarrow{3}$ D ✓

C $\xrightarrow{3}$ D $\xrightarrow{1}$ A $\xrightarrow{2}$ E ✓



Si se elimina la clave 21 y se "promociona" la clave más a la izquierda del subárbol derecho, el árbol quedará desbalanceado.



Obs:
 idem con 25
 si promociona
 con el otro
 entero